Facial Emotion Recognition with a Custom CNN

Computer Vision Assignment

Falak Baweja & Tejas Khanna (Lyari Task Force)

Contents

1 Introduction

This report documents the design, training, and evaluation of a custom convolutional neural network (CNN) built from scratch for the task of facial emotion recognition on the FER-2013 dataset. The objective is to classify a 48×48 grayscale face image into one of seven discrete emotion categories: *angry, disgust, fear, happy, neutral, sad,* and *surprise*.

In line with the project brief, the architecture is implemented entirely from first principles — no pre-trained weights and no high-level model wrappers such as `torchvision.models` are used. The focus is on understanding *why* each architectural and training decision is made, with every choice grounded in a specific finding from the literature. The dataset is the FER-2013 benchmark, which contains approximately 28,709 training images and 7,178 test images (further split into a public validation set and a private test set), and is well known for being noisy and class-imbalanced. These challenges, particularly the severe under-representation of the *disgust* class, are addressed explicitly through oversampling and data augmentation. The final model, `CustomEmotionCNN`, contains roughly 2.37 million trainable parameters and achieves a test accuracy of **64.47%** on the FER-2013 PrivateTest split.

2 Architecture

2.1 Inspiration

The architecture draws primarily from the **VGGNet** paper by Simonyan and Zisserman (ICLR 2015), which established that depth, achieved through stacks of small 3×3 convolutions, is the single most important factor for strong image-recognition performance. Three ideas were carried directly into this project: the exclusive use of 3×3 filters stacked in pairs (so that two consecutive 3×3 convolutions match the receptive field of a single 5×5 but introduce an additional non-linearity with fewer parameters), the progressive doubling of channel width after each pooling stage ($32 \rightarrow 64 \rightarrow 128 \rightarrow 256$, scaled down from VGG's

$64 \to 512$ to suit the 48×48 input), and the use of 2×2 max-pooling for spatial downsampling between blocks. The second paper that shaped the design is the **Batch Normalization** paper by Ioffe and Szegedy (ICML 2015). Batch Normalization is applied after every convolutional layer (before the ReLU activation) in order to reduce internal covariate shift, accelerate convergence, dampen sensitivity to weight initialisation, and provide a mild regularisation effect that complements Dropout in the fully-connected head.

2.2 Architecture

The model, named `CustomEmotionCNN`, consists of six convolutional layers organised into four VGG-style blocks, followed by a three-layer fully-connected classification head. Each block uses 3×3 convolutions with `padding=1`, and the deeper blocks (3–4 and 5–6) stack two convolutions before pooling, mirroring VGGNet's paired-convolution pattern. Batch Normalisation is applied at the end of each block (specifically after `conv1`, `conv2`, `conv4`, and `conv6`), and Dropout with $p = 0.5$ is used in both fully-connected layers as the primary regulariser. The full layer-by-layer breakdown is given in Table 1. The total parameter count is **2,373,191**.

Table 1: Layer-by-layer architecture of `CustomEmotionCNN`.

Layer / Block	**Output Shape**	**Details**
Input	$1 \times 48 \times 48$	Grayscale image
Conv1 + BN1 + ReLU + Pool1	$32 \times 24 \times 24$	Conv 3×3, 32 filters, pad=1; MaxPool 2×2
Conv2 + BN2 + ReLU + Pool2	$64 \times 12 \times 12$	Conv 3×3, 64 filters, pad=1; MaxPool 2×2
Conv3 + ReLU	$128 \times 12 \times 12$	Conv 3×3, 128 filters, pad=1 (no pool)
Conv4 + BN3 + ReLU + Pool3	$128 \times 6 \times 6$	Conv 3×3, 128 filters, pad=1; MaxPool 2×2
Conv5 + ReLU	$256 \times 6 \times 6$	Conv 3×3, 256 filters, pad=1 (no pool)
Conv6 + BN4 + ReLU + Pool4	$256 \times 3 \times 3$	Conv 3×3, 256 filters, pad=1; MaxPool 2×2
Flatten	2304	$256 \times 3 \times 3 = 2304$
FC1 + ReLU + Dropout(0.5)	512	Linear($2304 \to 512$)
FC2 + ReLU + Dropout(0.5)	128	Linear($512 \to 128$)
FC3 (output)	7	Linear($128 \to 7$), raw logits

The output of `FC3` is left as raw logits because `nn.CrossEntropyLoss` in PyTorch applies a log-softmax internally, avoiding redundant computation and improving numerical stability.

3 Data Augmentation and Reasoning

The FER-2013 dataset is both noisy and heavily imbalanced. The *disgust* class contains only ∼436 training samples — roughly ten times fewer than *happy* (∼7,215). To prevent the model from learning to simply ignore this minority class, the disgust samples were replicated nine additional times (ten times total) at the dataset level, bringing them into the same order of magnitude as the other classes. This deliberate oversampling strategy was chosen over a weighted loss because it integrates cleanly with the existing `ImageFolder` pipeline and allows the augmentation transforms to act on each replicated copy independently, producing genuine variation rather than identical duplicates.

The training-time augmentation pipeline was implemented via `torchvision.transforms.Compose` and applied only to the training set. Validation and test sets received nothing more than `Grayscale` → `Resize` → `ToTensor`, ensuring unbiased evaluation. The training augmentations are summarised in Table 2.

Table 2: Training-time augmentations and their rationale.

Augmentation	**Rationale**
Grayscale conversion	FER-2013 is inherently single-channel; enforces consistent 1-channel input.
Resize to 48×48	Standardises input dimensions for the network.
RandomHorizontalFlip ($p = 0.5$)	Human faces are roughly bilaterally symmetric, so flipping doubles the effective dataset size without altering the emotion label.
RandomRotation ($\pm 15°$)	Simulates natural head tilt and minor pose variation, improving robustness to real-world inputs.
ColorJitter (brightness = 0.2)	Perturbs image intensity to simulate varying lighting conditions, even within grayscale.

4 Results

4.1 Final Accuracy

The model was trained for 40 epochs using the Adam optimiser (`lr`= 10^{-3}, `weight_decay`= 10^{-4}) with a `ReduceLROnPlateau` scheduler (`factor`= 0.5, `patience`= 2) and a batch size of 64. The loss function was standard cross-entropy. Final performance on the validation (PublicTest) and test (PrivateTest) splits is given in Table 3, and per-class test accuracy is given in Table 4.

Table 3: Final loss and accuracy on the validation and test sets.

Metric	**Validation Set**	**Test Set (PrivateTest)**
Accuracy	63.47% (epoch 39)	**64.47%**
Loss	1.0513	1.0053

Table 4: Per-class accuracy on the PrivateTest set.

Emotion Class	**Test Accuracy**
Happy	87.8%
Surprise	77.2%
Neutral	66.1%
Disgust	65.5%
Angry	56.2%
Sad	50.3%
Fear	37.1%

4.2 Loss Curves

Training loss decreased monotonically from 1.76 at epoch 1 to 0.66 at epoch 40, while validation loss fell from 1.67 to approximately 1.05, plateauing around epochs 27–28. Validation accuracy climbed steadily from 36.9% at epoch 1 to a peak of 63.47% at epoch 39. The most pronounced accuracy jumps (epochs 26–28: 60.66% $\rightarrow$ 62.52%) coincide with automatic learning-rate reductions triggered by the scheduler, confirming that the LR-on-plateau strategy was actively contributing to convergence. The moderate gap between training and validation loss in the final epochs indicates mild overfitting, partially controlled by Dropout and weight decay.

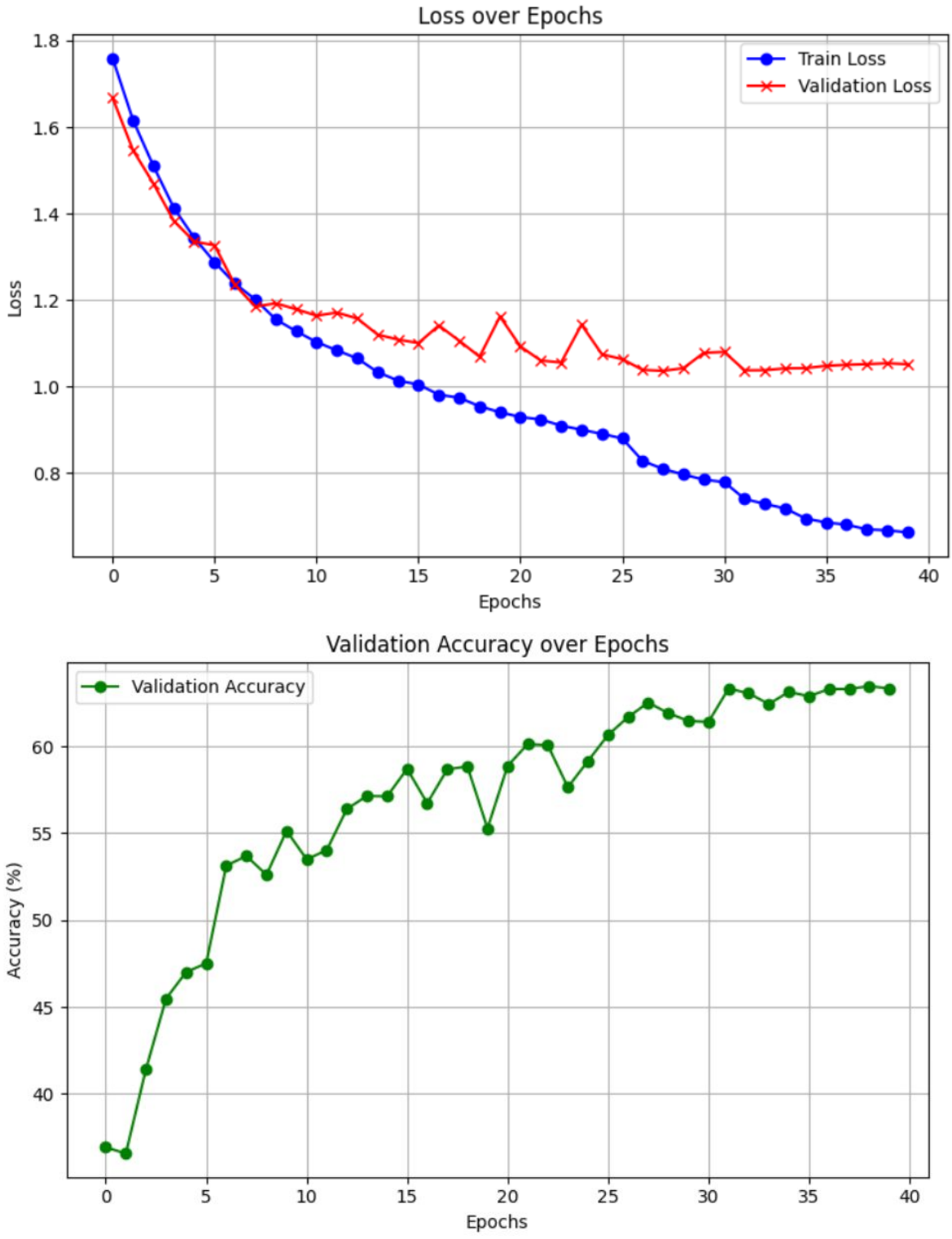

Figure 1: Training/validation loss (left) and validation accuracy (right) over 40 epochs.

4.3 Confusion Matrix

The confusion matrix on the PrivateTest set (Figure 2) reveals a clear pattern. *Happy* (87.8%) and *Surprise* (77.2%) are the easiest classes, owing to their distinctive visual signatures — wide smiles, raised eyebrows, and open mouths. *Fear* is by far the most problematic class at 37.1%, and is frequently misclassified as *Sad* or *Neutral*, both of which share subtle, low-contrast facial cues with fear. This is a well-documented characteristic of FER-2013 and reflects genuine label ambiguity in the dataset rather than a pure modelling failure. The *Disgust* class, despite being the rarest in the original data, reaches 65.5% accuracy — direct evidence that the 10× oversampling strategy succeeded in giving the model enough signal to learn the minority class.

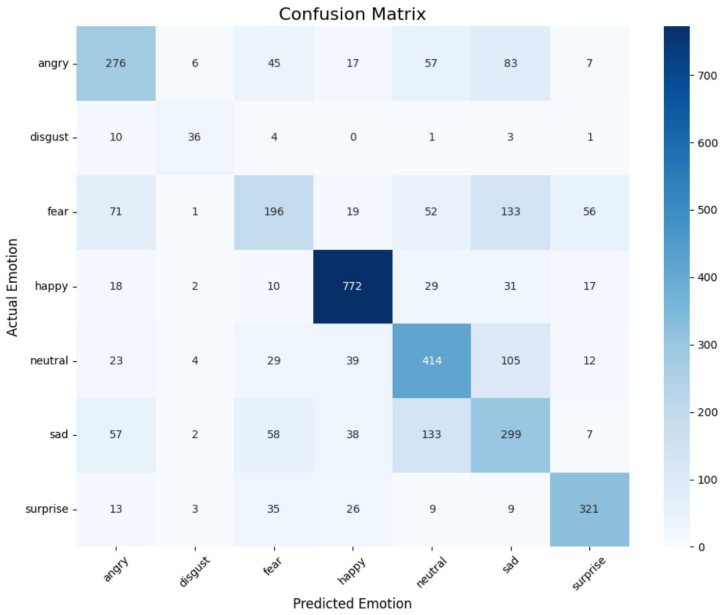

Figure 2: Confusion matrix on the FER-2013 PrivateTest set.

5 Key Learnings: Successes, Challenges, and Future Improvements

Several choices contributed clearly to the final result. Adding **weight decay** (10^{-4}) to the Adam optimiser provided a useful L2 regularisation signal that worked alongside Dropout to control the mild overfitting visible in the loss curves. Settling on **40 epochs** as the training budget turned out to be close to optimal — shorter runs of around 15 epochs left the model clearly under-trained (validation accuracy still climbing past 55%), while pushing the training to 100 epochs caused validation loss to drift upward as the model began memorising the noisy FER-2013 labels. The 10× **oversampling of the disgust class** also proved essential: it took the minority class from being effectively invisible to the loss function to achieving 65.5% per-class accuracy, comparable to several of the better-represented classes. The main remaining weakness is **Fear**, which sits at 37.1% and is consistently confused with sadness and neutrality — a limitation rooted as much in dataset label noise as in the model itself. Given more time, the most promising next steps would be to introduce **ResNet-style residual connections** to allow deeper stacking without vanishing-gradient degradation, and to explore a **Transformer-based architecture** (e.g., a Vision Transformer or hybrid CNN-Transformer model) to better capture the long-range spatial relationships between facial regions that distinguish subtly different emotions.

References

1. Simonyan, K., & Zisserman, A. (2015). *Very Deep Convolutional Networks for Large-Scale Image Recognition*. ICLR 2015. arXiv:1409.1556.

2. Ioffe, S., & Szegedy, C. (2015). *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift*. ICML 2015. arXiv:1502.03167.